THESIS - EE255401

AUTONOMOUS VEHICLE NAVIGATION SYSTEM USING MACHINE LEARNING AND GRAPH BASED SLAM

AZZAM WILDAN MAULANA
NRP 6022241101

SUPERVISOR
Prof. Dr. Ir. Mauridhi Hery Purnomo, M.Eng.
Dr. Ahmad Zaini, S.T., M.Sc.
Dr. Rudy Dikairono, S.T., M.T.

MASTER PROGRAM
DEPARTMENT OF ELECTRICAL ENGINEERING
FACULTY OF INTELLIGENT ELECTRICAL AND INFORMATICS TECHNOLOGY
INSTITUT TEKNOLOGI SEPULUH NOPEMBER
SURABAYA
2025

This page is intentionally left blank

VALIDITY SHEET
THESIS PROPOSAL

Judul	:	AUTONOMOUS VEHICLE NAVIGATION SYSTEM USING MACHINE LEARNING AND GRAPH BASED SLAM
Oleh	:	Azzam Wildan Maulana
Nrp	:	6022241101

Has Been Presented On

Day	:	Thursday
Date	:	26 June 2025
Place	:	B211 Room

Examiner

1. Dr. Arief Kurniawan, S.T., M.T.
   Nip :197409072002121001

2. Dr. Diah Puspito Wulandari, S.T., M.Sc.
   Nip :198012192005012001

3. Dr. Supeno Mardi Susiki Nugroho,S.T.,M.T.
   Nip :197003131995121001

Supervisor

1. Prof. Dr. Ir. Mauridhi Hery Purnomo, M.Eng.
   Nip :195809161986011001

2. Dr. Ahmad Zaini, S.T., M.Sc.
   Nip :197504192002121003

3. Dr. Rudy Dikairono, S.T., M.T.
   Nip :198103252005011002

This page is intentionally left blank

PREFACE

Praise and gratitude to the presence of Allah SWT. for all His grace, the author was able to complete this research with the title **AUTONOMOUS VEHICLE NAVIGATION SYSTEM USING MACHINE LEARNING AND GRAPH BASED SLAM**. This research was written as a requirement for completing a master's degree at the Department of Electrical Engineering, Institut Teknologi Sepuluh Nopember.
The author also expresses great thanks to:

- Parents and family for their encouragement and support during the author's study.
- One girl who has always been there for the author, providing support and motivation during the author's study.
- My friends especially AWM and The Dangereous Family for supporting the author during the study.

Surabaya, June 2025

The Author

This page is intentionally left blank

AUTONOMOUS VEHICLE NAVIGATION SYSTEM USING MACHINE LEARNING AND GRAPH BASED SLAM

By	:	Azzam Wildan Maulana
Student Identity Number	:	6022241101
Supervisor	:	1. Prof. Dr. Ir. Mauridhi Hery Purnomo, M.Eng.
		2. Dr. Ahmad Zaini, S.T., M.Sc.
		3. Dr. Rudy Dikairono, S.T., M.T.

ABSTRACT

Icar is an autonomous vehicle developed by Institut Teknologi Sepuluh Nopember (ITS) to navigate the campus environment autonomously. Its previous navigation system relied on pose estimation through sensor fusion between odometry and the Global Navigation Satellite System (GNSS). However, GNSS signals are prone to degradation in environments with obstacles such as tall buildings and trees, while odometry suffers from wheel slippage, encoder drift, and inertial sensor errors. These limitations lead to unreliable localization, potentially causing Icar to deviate from its intended trajectory.

To address these challenges, this research proposes an enhanced navigation system by integrating a camera and LiDAR. Road detection is performed using a deep-learning-based semantic segmentation model, enabling Icar to distinguish road surfaces from non-road areas in real time. Pose refinement is achieved through graph-based optimization within a SLAM framework, incorporating depth camera and LiDAR data aligned using the Iterative Closest Point (ICP) algorithm. The proposed Graph-Based SLAM significantly improves pose estimation accuracy, reducing the maximum error from 2.24 meters to 0.10 meters and improving precision from 0.69 meters to 0.03 meters.

Furthermore, the Road Segmentation module achieves a best IoU of 0.7928, with temporal filtering implemented through GRU embedding and frame-by-frame low-pass filtering to enhance segmentation stability. Overall, the integration of SLAM and machine learning provides a more robust and GNSS-independent navigation solution, improving Icar's reliability when operating in semi-structured or dynamic environments.

Keyword: Autonomous vehicle, Icar, GPS, Odometry, Pose Estimation, Camera, Semantic Segmentation, SLAM, ICP, Graph-Based Optimization, Deep Learning

This page is intentionally left blank

TABLE OF CONTENTS

This page is intentionally left blank

LIST OF FIGURES

This page is intentionally left blank

LIST OF TABLES

This page is intentionally left blank

NOMENKLATUR

Symbol / Acronym	Description
x_{waypoint}	X-coordinate of the waypoint from the predefined trajectory
y_{waypoint}	Y-coordinate of the waypoint from the predefined trajectory
x_{position}	Current X-coordinate of the vehicle
y_{position}	Current Y-coordinate of the vehicle
$\theta_{\text{direction}}$	Heading angle toward the waypoint
$\theta_{\text{orientation}}$	Current orientation of the vehicle
v_{target}	Target velocity of the vehicle
v_{max}	Maximum velocity allowed
δ_{target}	Target steering angle based on bicycle model
L	Distance between the front and rear wheels
$D_{\text{lookahead}}$	Lookahead distance for path tracking
GNSS	Global Navigation Satellite System
SLAM	Simultaneous Localization and Mapping
ICP	Iterative Closest Point
IMU	Inertial Measurement Unit
EKF	Extended Kalman Filter
GTSAM	Georgia Tech Smoothing and Mapping
DoF	Degrees of Freedom
RTK	Real-Time Kinematic (high-precision GNSS)
DGNSS	Differential GNSS
ML	Machine Learning
CNN	Convolutional Neural Network
Fast-SCNN	Fast Semantic Segmentation Convolutional Neural Network
SVD	Singular Value Decomposition
LIDAR	Light Detection and Ranging
PCL	Point Cloud Library
Odometry	Estimation of position based on wheel rotations and IMU data
Pose	A combination of position and orientation in 2D or 3D space
Yaw	Rotation around the vertical (z) axis
Kalman Filter	Recursive algorithm for optimal state estimation
Multipath	GNSS signal distortion caused by reflection or obstruction
Trajectory	A planned path for the robot to follow
Feature Matching	Process of aligning keypoints across frames or sensors

This page is intentionally left blank

CHAPTER 1
INTRODUCTION

1.1 Background

Icar is an autonomous vehicle developed by Institut Teknologi Sepuluh Nopember (ITS). Icar is designed to operate automatically and navigate throughout the ITS campus. Its control system relies on pose estimation (i.e., position and orientation) derived from odometry and sensor fusion with the Global Navigation Satellite System (GNSS). The estimated pose is then used to guide Icar toward a predetermined destination.

The pose of Icar is determined through the combination of odometry and GNSS data. Odometry estimates pose based on wheel rotations, augmented by a gyroscope to obtain orientation. GNSS, on the other hand, estimates pose based on satellite signals. These two sources are fused using the Extended Kalman Filter algorithm to achieve higher accuracy in pose estimation [28].

However, the data provided by GNSS is not always fully reliable. Several environmental conditions, such as tall buildings or dense tree canopies, can obstruct satellite signals. This may cause a phenomenon known as multipath, in which satellite signals reach the receiver via multiple indirect paths, resulting in errors in position and orientation estimation [11].

Errors can also arise from the odometry system due to factors such as inaccurate wheel travel distance, incorrect wheel rotation angle calculations, or faulty IMU sensor readings. These inaccuracies may result from differences in wheel diameters, uneven tire pressure, or wheel slippage. Moreover, errors in wheel rotation angle measurements may originate from malfunctioning encoder sensors [3].

When errors occur in pose estimation, Icar may deviate from its intended trajectory. This can lead to safety concerns, such as collisions with obstacles or deviation from designated transportation lanes. Therefore, an additional navigation system is required to enhance pose accuracy and ensure safe

autonomous navigation.

A promising enhancement is the integration of a camera with LIDAR, which provides visual and depth information to detect roads. This road detection process employs a machine learning algorithm—specifically, semantic segmentation—trained on labeled datasets to distinguish between road and non-road areas [23].

Beyond road detection, the camera and LIDAR are also used to refine pose estimation through graph-based optimization. Furthermore, a LIDAR sensor is incorporated and processed using the Iterative Closest Point (ICP) algorithm to further improve localization accuracy. Together, these methods form a Simultaneous Localization and Mapping (SLAM) system [24], which is expected to potentially replace the role of GNSS entirely.

1.2 Formulation of the Problem

The main issue with Icar lies in its navigation system's heavy dependence on GNSS. When GNSS signal errors occur, or when odometry is affected by slippage or IMU drift, the resulting pose estimation becomes inaccurate. Consequently, Icar may deviate from its intended path, increasing the risk of collisions or deviation from public transportation routes.

1.3 Objective

The objective of this research is to develop a robust navigation system for Icar that is capable of correcting pose estimation errors caused by GNSS inaccuracies or odometry drift. This system incorporates a camera and LIDAR to perform both road detection and pose correction. The goal is to enable Icar to navigate more accurately and reliably in dynamic environments.

1.4 Scope and Limitations

This study focuses exclusively on Icar, an autonomous vehicle developed by ITS. The scope is limited to the development and implementation of a navigation system that utilizes a camera and LIDAR. This research does not cover the electronic or mechanical aspects of Icar's control system.

1.5 Contribution

The expected contribution of this research is to provide a reference framework for enhancing the accuracy and safety of autonomous vehicle navigation systems. By integrating advanced perception and localization technologies, Icar is expected to achieve improved trajectory tracking and obstacle avoidance, supporting its intended operation in complex urban environments.

This page is intentionally left blank

CHAPTER 2

LITERATURE REVIEW

To support this research, several relevant theories are required as references and conceptual foundations. This will help ensure the research is well-directed and aligned with existing scientific knowledge.

2.1 Odometry

Odometry is an algorithm used to estimate how far a robot has traveled. In a 2D pose system, the robot's movement is represented as x, y, yaw. The simplest form of 2D odometry combines motor rotation data with orientation data from an Inertial Measurement Unit (IMU). By applying forward kinematics to this sensor data, the robot's pose can be estimated relative to its initial position [28].

2.2 Bicycle Model

The bicycle model is a simplified mathematical representation of vehicle dynamics. It models a vehicle as a two-wheel system—one at the front and one at the rear—mimicking the dynamics of a bicycle. This model is widely used in vehicle control and trajectory planning due to its simplicity and ability to capture basic vehicle behavior [22]. In robotics and electrical engineering, the bicycle model is commonly used to develop navigation and motion control algorithms for autonomous ground vehicles.

$$\delta_{\text{target}} = \tan^{-1}\left(\frac{2 \cdot L \cdot \sin(\theta_{\text{direction}})}{D_{\text{lookahead}}}\right) \tag{2.1}$$

where δ_{target} is the target steering angle, L is the distance between the front and rear wheels, $\theta_{\text{direction}}$ is the heading angle, and $D_{\text{lookahead}}$ is the lookahead distance.

The main advantage of the bicycle model is its applicability in control and simulation algorithms such as PID, LQR, and MPC. Many autonomous

vehicle systems use it as a foundational model for motion prediction and path following. For instance, it enables calculation of the optimal steering angle required for a vehicle to accurately and stably follow a predefined trajectory [20].

2.3 GNSS

The Global Navigation Satellite System (GNSS) is a satellite-based system that provides highly accurate global information on position, velocity, and time. GNSS encompasses multiple satellite navigation systems, including GPS (USA), GLONASS (Russia), Galileo (EU), and BeiDou (China), which can function independently or in combination [18]. In electrical engineering and robotics, GNSS plays a critical role in navigation and positioning, particularly for mobile robots operating in open environments.

GNSS operates by measuring the time required for satellite signals to reach a receiver. A GNSS receiver can compute a 3D position using signals from at least four satellites. Techniques such as Differential GNSS (DGNSS) and Real-Time Kinematic (RTK) improve accuracy by applying corrections from fixed reference stations [9]. These techniques are especially beneficial in applications like precision agriculture, drones, and autonomous vehicles.

GNSS can be integrated with other systems such as Inertial Navigation Systems (INS) to enhance reliability in environments where satellite signals are blocked, such as under tree canopies or between buildings [7]. Integration with sensors like IMUs, LiDARs, and cameras also enhances performance in SLAM systems.

Beyond navigation, GNSS contributes to control systems, trajectory planning, and multi-robot coordination. With ongoing advances in satellite infrastructure and error correction algorithms, GNSS remains a cornerstone for the development of efficient and precise outdoor robotic systems.

2.4 Kalman Filter

The Kalman Filter is a powerful linear estimation algorithm used to predict and update the state of dynamic systems based on noisy sensor

data. Introduced by Rudolf E. Kalman in 1960, it enables accurate real-time estimation of variables such as position and velocity [8]. In robotics and control systems, the Kalman Filter is instrumental in fusing data from multiple sensors such as GNSS, IMUs, and wheel encoders.

The algorithm consists of two stages: prediction and update. During the prediction stage, the system state is estimated using a motion model. In the update stage, the estimate is refined based on new sensor measurements. This iterative process results in increasingly accurate state estimations [26].

2.5 SLAM

Simultaneous Localization and Mapping (SLAM) is a foundational technology in autonomous robotics. It allows a robot to construct a map of an unknown environment while simultaneously estimating its own position within that map. SLAM is particularly valuable in GPS-denied environments and is widely used in autonomous vehicles, drones, and mobile robots [4].

2.6 Graph-Based SLAM

SLAM techniques can be divided into two broad categories: filter-based SLAM (e.g., Extended Kalman Filter SLAM) and graph-based SLAM. In filter-based SLAM, the robot's pose and landmark positions are treated as stochastic variables updated recursively. In contrast, graph-based SLAM builds a graph where nodes represent robot poses and landmarks, and edges represent spatial constraints derived from sensor observations [24].

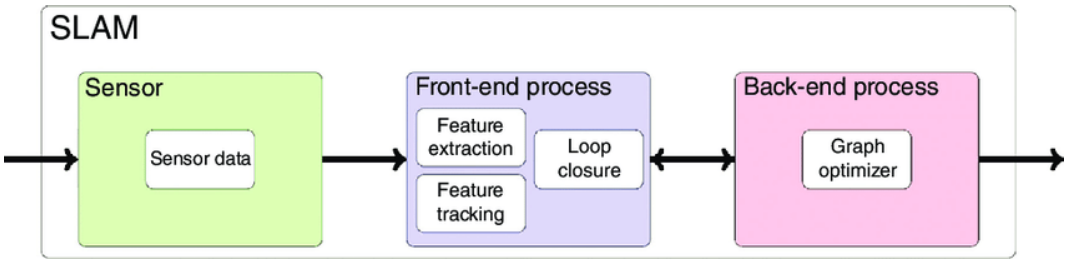

Figure 2.1. Basic Graph-Based SLAM Architecture [27]

There are three main processes that can be seen from figure 2.1: The first process is sensor data, it means like acquiring data from sensors such

as cameras, LIDAR, or IMU. The second process is front end process, The front end process is responsible for extracting features from the sensor data and finding a loop closure. The last process is back end process, the back end process is responsible for optimizing the graph to find the best estimate of the robot's trajectory and the map of the environment [27].

The optimization in Graph-based SLAM can be efficiently implemented using frameworks like GTSAM (Georgia Tech Smoothing and Mapping), a C++ library that applies nonlinear optimization over factor graphs. Factor graphs represent probabilistic relationships among variables such as odometry, landmarks, and sensor observations, and allow for efficient optimization of robot trajectories [5].

2.7 RTAB-Map

RTAB-Map (Real-Time Appearance-Based Mapping) is a graph-based SLAM framework designed for real-time applications in robotics. It integrates various sensors, including cameras, LIDARs, GPS, IMUs, and Odometry, to build a map of the environment while simultaneously localizing the robot within that map. RTAB-Map is particularly effective in large-scale environments and can handle loop closures and dynamic changes in the environment [14].

Stereo cameras are widely used in robotics due to their ability to generate real-time depth maps without requiring active sensors like LiDAR. They are especially useful in lightweight and power-efficient robotic platforms such as drones and small autonomous vehicles [12].

2.8 Machine Learning

Machine learning (ML) is a subset of artificial intelligence that focuses on creating algorithms capable of learning from data and improving performance without being explicitly programmed [19]. In robotics and electrical engineering, ML is essential for building intelligent systems that can recognize patterns, make decisions, and adapt to changing environments.

ML methods are typically categorized into three types: supervised learn-

ing, unsupervised learning, and reinforcement learning [6]. Supervised learning uses labeled datasets for tasks like classification and regression. Unsupervised learning deals with unlabeled data for clustering or dimensionality reduction. Reinforcement learning enables systems to learn optimal behaviors through reward-based interactions with the environment.

A key ML technique in robotic perception is semantic segmentation, which classifies each pixel in an image into predefined categories. This technique is widely used for road detection, scene understanding, and navigation.

2.9 Semantic Segmentation

Fast-SCNN (Fast Semantic Segmentation Convolutional Neural Network) was designed for real-time performance on resource-constrained devices, making it ideal for embedded systems like autonomous vehicles. It combines a lightweight encoder-decoder structure with efficient components such as depthwise separable convolutions and a streamlined feature fusion strategy. These design choices enable Fast-SCNN to achieve a good balance between accuracy and speed, ensuring that Road Segmentation can be performed in real time without requiring GPU-level hardware.

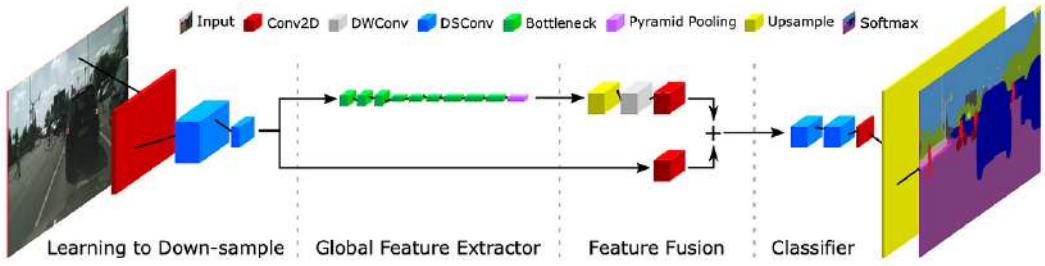

Figure 2.2. Fast SCNN Architecture [21]

Fast-SCNN consists of four main modules: Learning to Downsample, Global Feature Extractor, Feature Fusion Module, and Classifier. The network first reduces the spatial dimensions of the input image while increasing feature richness, then extracts global context features and fuses them with fine spatial details. This fused representation is finally classified at the pixel level to

distinguish road and non-road areas. In the Icar system, this segmented output supports safe navigation by identifying drivable regions based on real-time camera input [21].

2.10 Iterative Closest Point (ICP)

Iterative Closest Point (ICP) is a widely used algorithm in known-map localization that aligns two 3D point clouds: one from the robot's current observation and one from a reference map [29]. The output is a transformation matrix that minimizes the difference between the two point sets.

The ICP algorithm operates in three main steps: first, for each point in the input point cloud, it finds the nearest point in the reference point cloud (using nearest-neighbor search). Second, it calculates the centroids (center of mass) of both point clouds and aligns them via translation. Third, it applies rotation—typically using Singular Value Decomposition (SVD)—to best align the clouds.

These steps are repeated iteratively until convergence, defined by either reaching a maximum number of iterations or achieving a sufficiently low alignment error. The final result is the transformation matrix that aligns the current observation with the reference map, enabling accurate pose estimation.

2.11 Binary Robust invariant scalable keypoints (BRISK)

BRISK Binary Robust invariant scalable keypoints is a real-time feature method that combines a novel scale-space FAST detector with a binary bit-string descriptor built from intensity-comparison sampling. Keypoints are detected across scales via a FAST-based pyramid, then each is described by a 512-bit string derived from Gaussian-smoothed intensity comparisons on concentric sampling rings, with an orientation normalization step for rotation invariance. Evaluated on standard benchmarks, BRISK matches the accuracy of heavy float descriptors like SURF and SIFT while running up to an order of magnitude faster, making it highly suited for resource-constrained, real-time vision applications [15].

CHAPTER 3

FULL SYSTEM OVERVIEW

3.1 Overview of the Previous Icar System

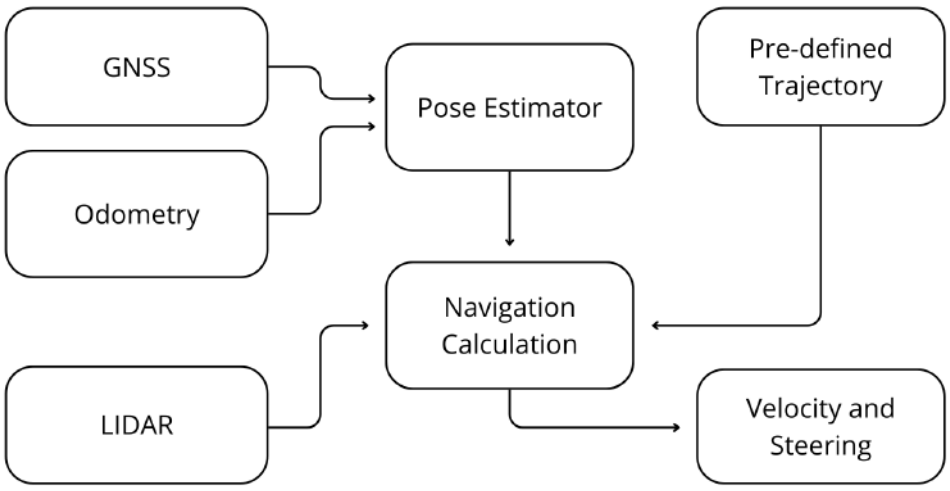

Figure 3.1. Block Diagram of the Previous Icar System with GNSS

Figure 3.1 illustrates the previous Icar system, which fully rely on GNSS and odometry for pose estimation. The system uses a Pre-defined Trajectory to determine the target speed and steering angle using the Bicycle Model

3.2 Overview of the Proposed System

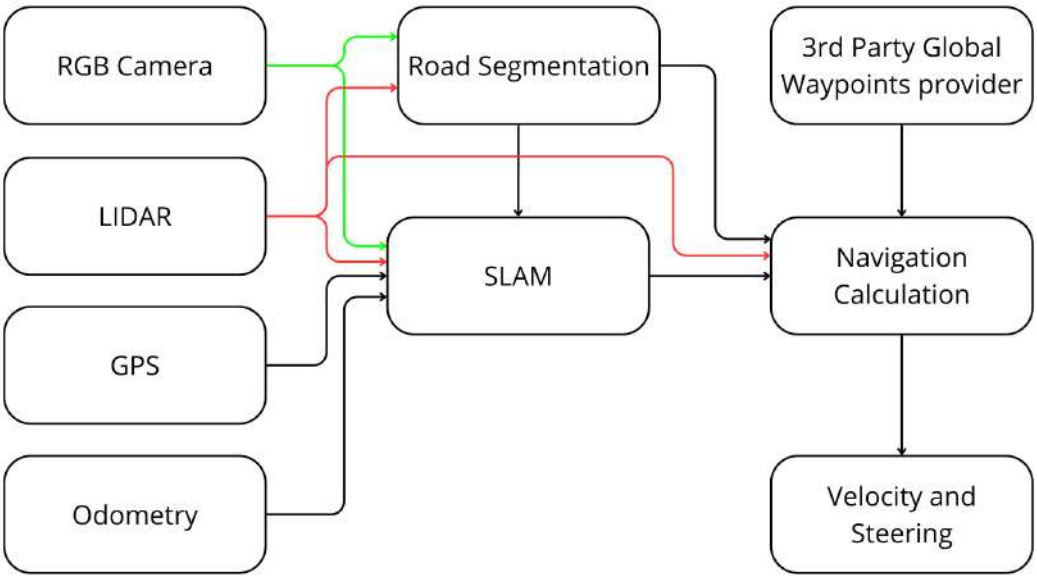

Figure 3.2. Block Diagram of the Proposed Icar System

Figure 3.2 illustrates the proposed Icar system, which integrates Graph Based SLAM for pose estimation. The system utilizes Sensor Fusion from LIDAR, Camera, GPS, and Wheel Velocity Sensor to enhance the accuracy and reliability of the vehicle's navigation capabilities.

CHAPTER 4

LOW LEVEL SYSTEM

4.1 Electrical System

The Electrical System of Icar consists of two main subsystems: the Sensor System and the Actuation System. The Sensor System is responsible for gathering data from the environment and the vehicle itself, while the Actuation System controls the movement and operation of the vehicle based on the calculated data.

4.1.1 Sensor System

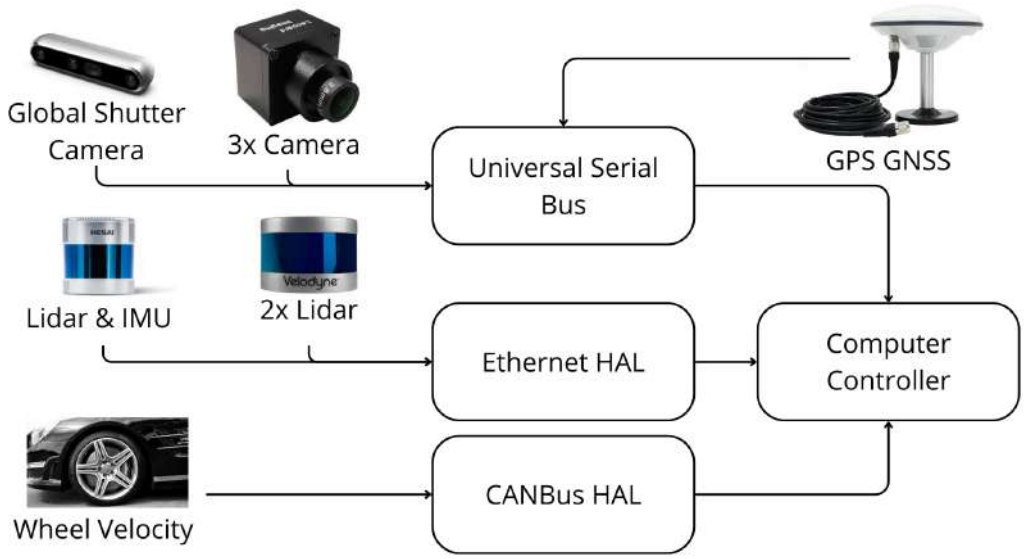

Figure 4.1. Block Diagram of Sensor System

Based on Figure 4.1, the Sensor System consists of several components, including LIDAR, Camera, GPS, and Wheel Velocity Sensor. The LIDAR sensors produce point cloud data and IMU data that interfaced with ethernet controller of computer. The Camera sensors produce image data that will be used for Road Segmentation, interfaced using USB controller of computer.

The GPS sensor produces the estimation of latitude and longitude for car that interfaced using USB. The Wheel Velocity Sensor captured by sniffing car's main CAN bus using CANbus HAL module.

4.1.2 Actuation System

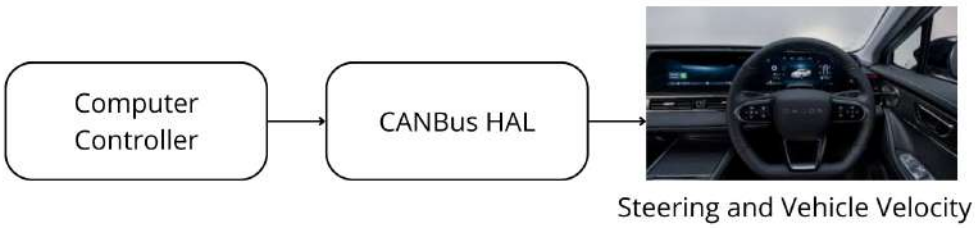

Figure 4.2. Block Diagram of Actuation System

Based on Figure 4.2, the Actuation System only proceeds Steering Angle and Velocity of vehicle. The commands are sent from the car's main CAN bus using CANbus HAL module. That technique is called Drive by Wire (DBW), where there is no physical connection between the driver and the actuator. The commands are generated using electrical signals from the computer to the car's main CAN bus.

4.2 Limitations

The limitations of the system are divided into two main categories: Sensor Limitations and Actuation Limitations. Each category has its own set of challenges that need to be addressed to ensure optimal performance of the autonomous vehicle.

4.2.1 Sensor Limitations

There are several limitations associated with the sensors used in the system. The limitations listed below:

- The noise measurement of GPS that can reach up to 2.5 meters in open sky condition, and can be worse in urban area with tall buildings or trees that can block the satellite signal.

- LIDAR Point Cloud has update rate limitation from 5 Hz to 20 HZ, which may affect the real-time performance for the higher level of system.

- There is 30 cm blindspot in front of car because of Lidar and Camera Mounting.

4.2.2 Actuation Limitations

There are several limitations associated with the actuation system used in the vehicle. The limitations listed below:

- The maximum car acceleration is $3.0\,\mathrm{m/s^2}$ based on ISO 11270, which limits the speed of response for velocity control.

- The maximum steering angle rate is $5\,\mathrm{deg/s}$ based on car's specification, which limits the speed of response for steering control.

- The maximum steering angle position is $\pm 300\,\mathrm{deg}$ based on car's specification, which limits the turning radius of the vehicle.

4.3 Low Level Program Architecture

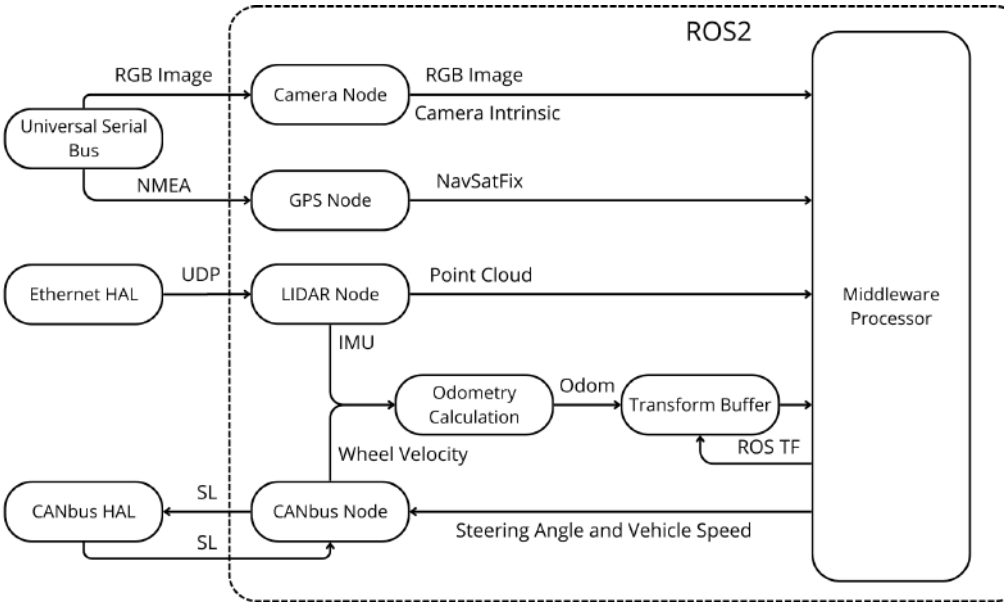

Figure 4.3. Block Diagram of Low Level Architecture

Based on Figure 4.3, the Low Level Program Architecture utilizes ROS2 as the framework for all processes. There are three main parts of Low Level Architecture, which are Sensor Data Acquisition, Odometry Calculation, and Frame Transformation.

4.3.1 Sensor Data Acquisition

Based on Figure 4.3, the Sensor Data Acquisition block is responsible for acquiring raw data from the LIDAR, camera, GPS, and wheel velocity sensor. For each sensor, we have a single independent ROS 2 node that publishes a standard ROS 2 message type. We use several standard ROS 2 message types in this part, such as `sensor_msgs/PointCloud2` for LIDAR point cloud data, `sensor_msgs/Image` for camera image data, `sensor_msgs/NavSatFix` for GPS data, and `std_msgs/Float32` for wheel velocity data.

4.3.2 Odometry Calculation

Based on Figure 4.3, the next step after Sensor Data Acquisition is Odometry Calculation. This block is responsible for computing the odometry from the wheel velocity sensor and the IMU data from the LIDAR. The

odometry is calculated using a differential-drive kinematic model, which takes into account the wheel velocities and the time interval between measurements. The resulting odometry data is then published as a `nav_msgs/Odometry` message type.

4.3.3 Frame Transformation

Based on Figure 4.3, there is a step called the transformation buffer. This part is responsible for managing the transformations between each sensor frame and the main frames, such as `base_link`, `odom`, and `map`. The diagram of the transformations between each frame is shown in Figure 4.4.

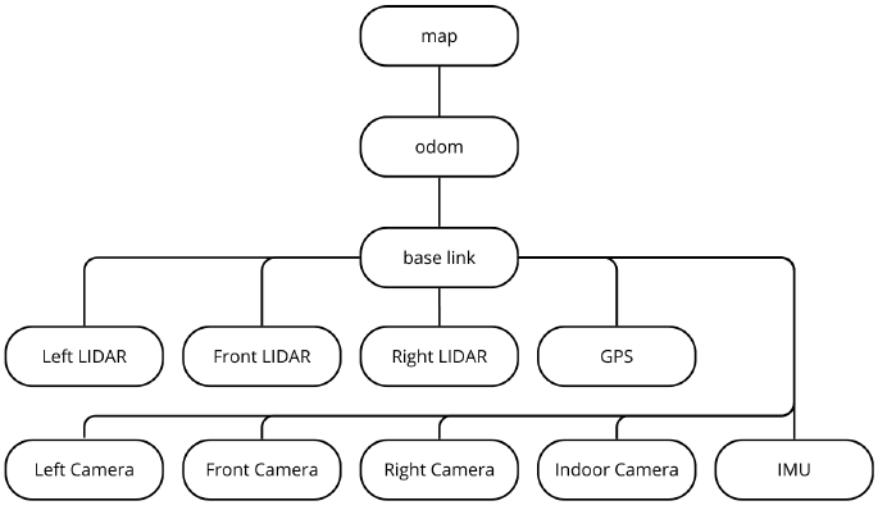

Figure 4.4. Transformation buffer

Figure 4.4 shows the transformations between each frame. The `base_link` frame is the main frame of the vehicle, which is located at the center of the vehicle. The `odom` frame represents the odometry, which is calculated from the wheel velocity sensor and IMU data from the LIDAR. The `map` frame is the global world frame that represents the global position of the vehicle. The transformations between frames are managed using the `tf2` library in ROS 2.

CHAPTER 5

MIDDLEWARE SYSTEM

5.1 Middleware System

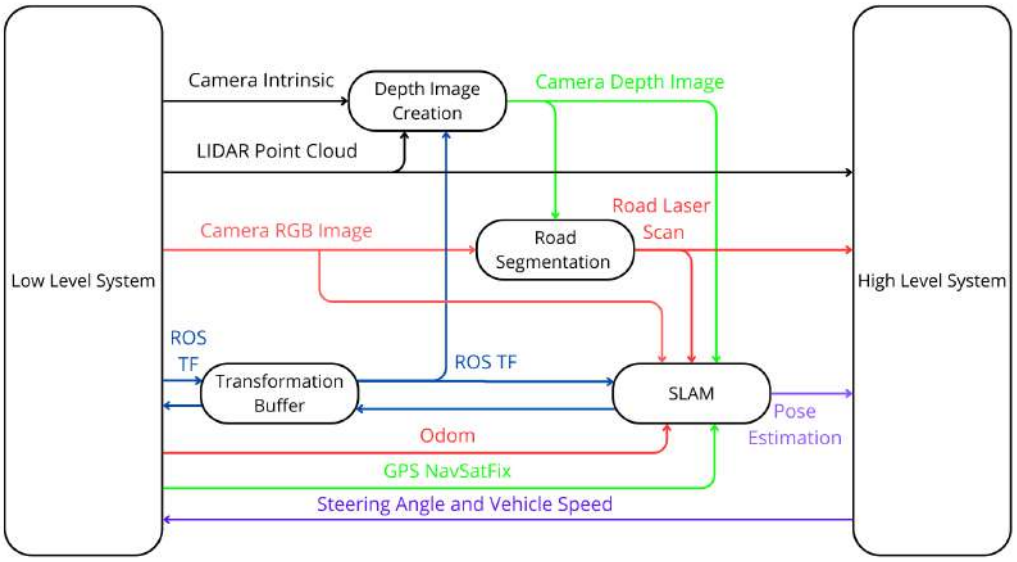

Figure 5.1. Block Diagram of Middleware System

Figure 5.1 shows the Middleware System that connects the Low Level Program Architecture with the High Level Program Architecture. There are three main parts in Middleware System, the first part is the Depth Image Creation, the second part is Road Segmentation using Machine Learning, and the third part is Graph Based SLAM for Pose Correction and Estimation.

5.2 Depth Image Creation

Based on Figure 5.1, the Depth Image Creation block is responsible for creating a depth image from the LIDAR point cloud data and camera intrinsic data. This process involves two main steps: LIDAR Concatenation and Post Filter for Depth Image.

5.2.1 LIDAR Concatenation

LIDAR Concatenation is the process of combining multiple LIDAR scans into a single point cloud. This is done to increase the density of the point cloud and improve the accuracy of the depth image. The concatenated point cloud is then projected onto the camera optical frame. The transformation lookup between the LIDAR frame and the camera frame can be obtained from the transformation buffer in the Low Level Program Architecture 4.4.

After transformed point cloud to the camera optical frame, we can calculate the depth image using the camera intrinsic parameters. Each point in the point cloud is projected onto the image plane using the pinhole camera model, and the depth value is calculated based on the distance from the camera to the point.

5.2.2 Post Filter for Depth Image

The raw depth image obtained from the previous step has some hole areas due to the sparse nature of the LIDAR point cloud. To address this issue, a post-filtering process is applied to fill in the holes and smooth the depth image. This process involves using morphological operations to fill in the holes by neighboring pixel values.

5.3 Machine Learning

Based on figure 5.1, another sub process of middleware is Road Segmentation. Road Segmentation is done using Machine Learning approach. The idea is to segment each pixel of the image into two classes, which are road and non-road. The segmentation result will be transformed base link frame using depth image from previous step. The output of this process is a laser scan msg that will be used by high level system to determine target velocity and steering angle.

5.3.1 Architecture Design

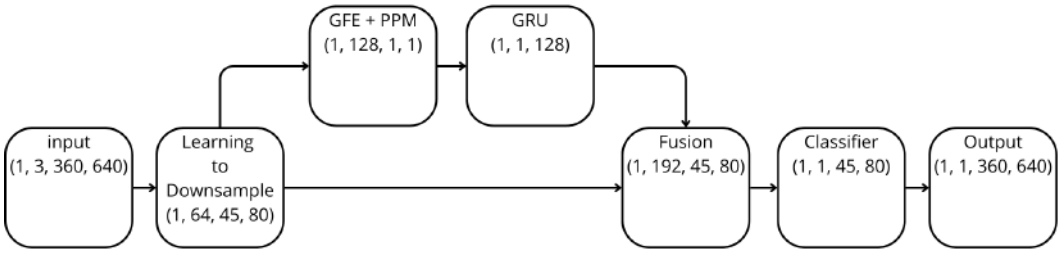

Figure 5.2. Machine Learning Architecture

Figure 5.2 visualize the architecture of our proposed new Machine Learning method. Based on Fast-SCNN, we proposed our new improvement method by adding GRU (Gated Recurrent Unit). The main idea of adding GRU to the existence of Fast-SCNN is to make a global context. This method introduces temporal consistency into the detection process.

5.3.2 Dataset Preparation

The preparation of dataset is crucial to Machine Learning process. The dataset only have one class called road class. The dataset image captured by driving the car through various road conditions, such as different lighting conditions, weather conditions, and road types. The dataset is then annotated manually to create ground truth labels for the road class.

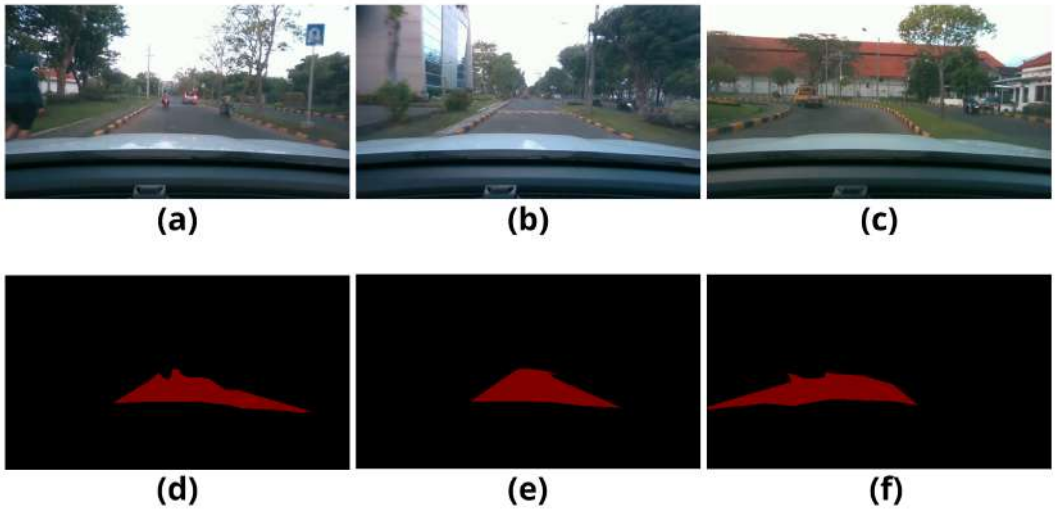

Figure 5.3. Datasets for Machine Learning

Figure 5.3 shows some example of 1000 images dataset that we used for training and validation. The image a, b, and c are the rgb input image, while the image d, e, and f are the corresponding ground truth labels for road class. The red segmented area in the ground truth labels represents the road class.

5.3.3 Model Training

The model training process is carried out using the prepared dataset. The problem of our dataset is imbalanced data for example on figure 5.3 in sub image b, there is a road bump that have same color and pattern with road barrier but we want to predict that it's a road too. To handle that imbalanced data we use the Tversky loss function during training. The Tversky loss function can handle imbalanced data and bunch of false positive cases [1]. The importance of handling false positive cases is to avoid the vehicle from misdetecting the road area as non-road area, which can lead to unsafe driving conditions.

Another technique that we use to improve the model performance is data augmentation. Data augmentation is done by applying various transformations to the input images, such as random gamma, random brightness, CLAHE, HSV limit, RGB shift, random shadow, random sunflare, ISO noise, Motion blur,

random tone curve, and random brightness contrast. These transformations help to increase the diversity of the training data and improve the model's ability to generalize to unseen data.

Another important aspect of the model training process is hyperparameter tuning. We experiment with different learning rates, batch sizes, and number of epochs to find the optimal combination that yields the best performance on the validation set. The model is trained using the Adam optimizer with an initial learning rate of 0.001 and a batch size of 12. The training is carried out for 400 epochs, with early stopping implemented to prevent overfitting.

Figure 5.2 shows that there's a GRU block after the feature extractor block. In the training process we disable saving the global context because our dataset were very random and not sorted yet.

5.3.4 Model Inference

After training process has done, the model was saved in pth format. That model deployed on Icar's PC that have embedded nvidia GPU with 8Gb of VRAM. Because the model is very lightweight, we don't need to convert it to onnx or another optimizer. We just load the model in pth format and doing computation in GPU.

5.3.5 Post Processing

There is an important step of Road Segmentation called Post Processing. There are step by step that we have done in post processing. The first step is ROI crop that will erase all unused pixel that will not be the road. The second step is morphological operation, the morphological operation is done to remove small noise and fill small holes in the segmented image. The third step is largest contour detection, the largest contour detection is done to find the largest road area in the segmented image. The fourth step is applying a low pass filter between time steps for every pixel to make the segmentation result more stable.

5.3.6 Final Transformation

The final step of Road Segmentation is Final Transformation. The final transformation is done by using the calculated depth image from 5.2 to transform the segmented image from camera frame to base link frame. The output of this process is a laser scan msg that will be used by high level system to determine target velocity and steering angle.

The first process of final transformation is doing a high pass filter on segmented area. The high pass filter is done to get the edge of the road area. After that, we perform a virtual laser scan from 0 degree to 180 degree in base link frame with 0.01 degree resolution. For every angle, we search for the closest pixel that has segmented area in the depth image. The distance from base link frame to that pixel is the range value for that angle in laser scan msg.

5.4 Graph Based SLAM

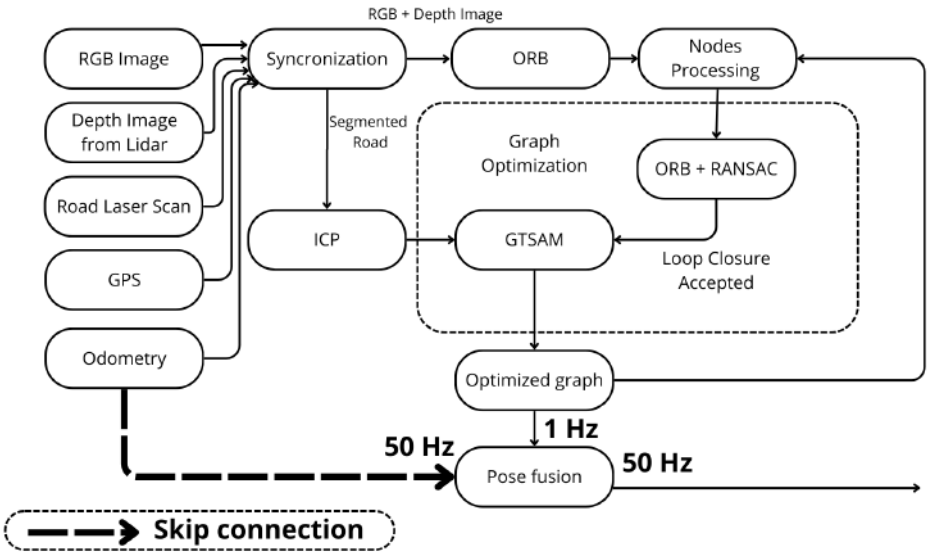

Figure 5.4. Block Diagram of Graph Based SLAM based on Extended RTAB-Map [17]

Figure 5.4 is the internal explanation of SLAM block in figure 5.1. The Graph Based SLAM is responsible for providing the best estimation of vehicle pose by fusing all sensor data from RGB image, Depth image, Road laser scan,

GPS, and Odometry from wheel encoder and IMU. The Graph Based SLAM consists of several main parts, which are Synchronization, Nodes Processing, Loop Closure Detection, Graph Optimization, and Pose Fusion.

The difference between our improved Graph Based SLAM with the previous Extended RTAB-Map [17] is the addition of GPS sensor to be a first guess for loop closure finding process. Another difference is doing ORB feature extraction instead of using GFTT because [10] said that ORB better than GFTT in outdoor environment.

5.4.1 Graph Based SLAM Mode

The Graph Based SLAM has two modes of operation, which are Mapping Mode and Localization Mode. In Mapping Mode, the Graph Based SLAM builds a map of the environment while simultaneously estimating the vehicle's pose. This mode is typically used during the initial exploration of an unknown environment. In Localization Mode, the Graph Based SLAM uses the previously built map to estimate the vehicle's pose without updating the map. This mode is typically used when the vehicle is operating in a known environment.

5.4.2 Synchronization

The first step of Graph Based SLAM is Synchronization. Based on [17], the synchronization strategy has been done using approximate time synchronization. The approximate time synchronization is done to synchronize all sensor data from RGB image, Depth image, Road laser scan, GPS, and Odometry from wheel encoder and IMU. The synchronized data is then used for the next step,

5.4.3 Nodes Processing

The first step of Nodes Processing is feature extraction of the image from camera. The feature extraction is done using ORB (Oriented FAST and Rotated BRIEF) [25] to extract keypoints and descriptors from the RGB image and Depth image. After the extraction done, the Node controller will check if the current node is new or revisited node. If the current node is a

new node, the node controller will add the new node to the graph. The new node will contain the RGB image, Depth image, Road laser scan, GPS, and Odometry data. If the current node is a revisited node, the node controller will proceed to the Loop Closure Detection step.

5.4.4 Loop Closure Detection

After all the feature extraction has done, the next step is to find loop closure candidates. By doing first guess using GPS data (LoopGPS=true), the loop closure will only search for candidate inside local radius searching area [13]. Like the method from [17], the loop closure detection is done using feature matching between the current node and the nodes inside the local radius searching area. The feature matching is done using ORB descriptors and RANSAC to filter out outliers. If the number of inliers is greater than a predefined threshold, the loop closure hypothesis is added.

5.4.5 Graph Optimization

After the loop closure hypothesis has been found, the next step is Graph Optimization. The graph optimization is done using GTSAM (Georgia Tech Smoothing and Mapping) [5] to optimize the graph and find the best estimation of vehicle pose by using the ICP constraint and odometry constraint. The graph optimization is done using the Levenberg-Marquardt algorithm to minimize the error between the predicted pose and the observed pose from the sensor data. The optimized graph is then used for the next step, which is Pose Fusion.

5.4.6 Pose Fusion

Like the method from [17], the Pose Fusion is done using an Extended Kalman Filter (EKF) to fuse the optimized pose from the Graph Optimization step with the odometry data from the wheel encoder and IMU. The fusion strategy use an odometry data as a velocity by differentiating the position data from odometry. The pose estimation from Graph Optimization is used as a measurement for position update in EKF. The output of the Pose Fusion is the best estimation of vehicle pose, which is then used by the High Level

Program Architecture for trajectory planning and control.

CHAPTER 6

HIGH LEVEL SYSTEM

6.1 High Level Program Architecture

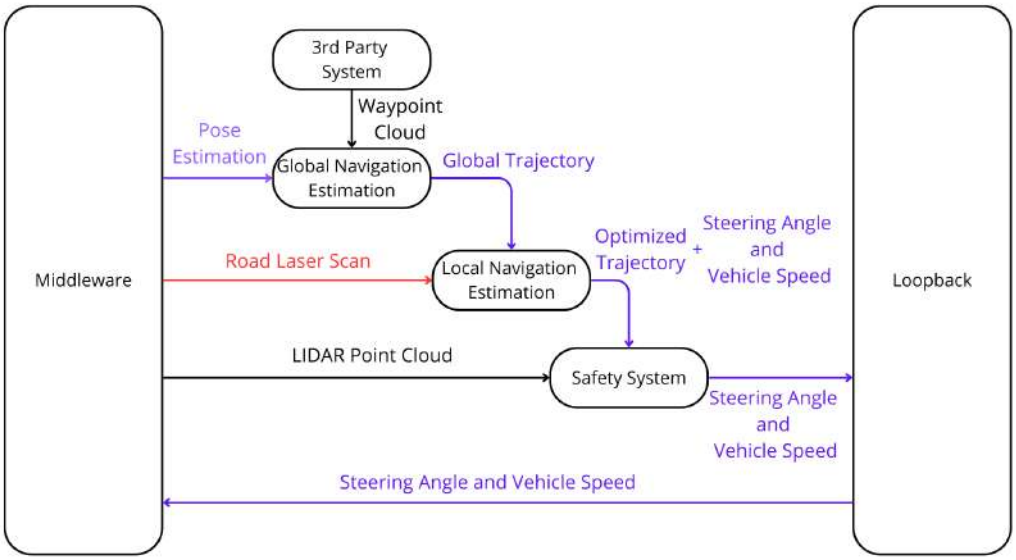

Figure 6.1. Block Diagram of High Level Architecture

Based on Figure 6.1, the High Level Program Architecture utilizes ROS2 as the framework for all processes. There are three main parts of High Level Architecture, which are Global Navigation Estimation, Local Navigation Estimation, and Safety System.

6.1.1 Global Navigation Estimation

The Global Navigation System is responsible for integrate with global trajectory from 3rd party application such as Google Maps or Waze. The 3rd party application will provide a global waypoints to system in latitude and longitude format. By using the final pose estimation from Graph Based SLAM as the starting point, the global waypoints will be converted to local waypoints in base link frame. The local waypoints will be used by Local

Navigation Estimation to plan the trajectory and control the vehicle.

6.1.2 Local Navigation Estimation

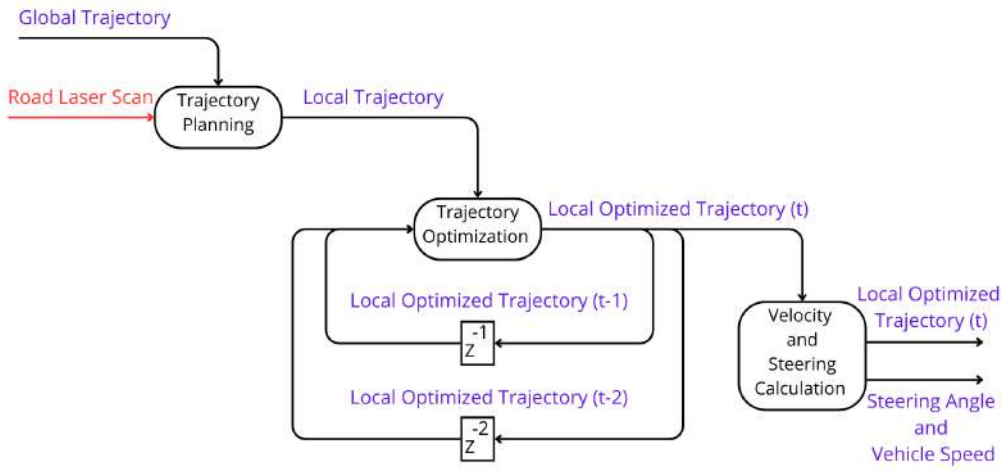

Figure 6.2. Block Diagram of Local Navigation System

Figure 6.2 shows the internal explanation of Local Navigation block in figure 6.1. The Local Navigation System divided into three main parts, which are Trajectory Planning, Trajectory Optimization, and Target Velocity and Steering Calculation.

6.1.2.1 Trajectory Planning

The most important part of Local Navigation System is Trajectory Planning. The Trajectory Planning is responsible for planning a safe and optimal trajectory from the current vehicle pose to the next local waypoint provided by Global Navigation Estimation. The Trajectory Planning uses the final pose estimation from Graph Based SLAM as the starting point and the local waypoints as the goal points. The planned trajectory will be used by Trajectory Optimization to optimize the trajectory for smoothness and feasibility.

The strategy used for Trajectory Planning is the Hybrid A* algorithm [16]. Hybrid A* was chosen because it operates in a continuous state space,

evaluating transitions in (x, y, θ) rather than discrete grid cells. This allows the planner to generate kinematically feasible paths that follow the non-holonomic constraints of an Ackermann steering vehicle. Unlike classical grid-based A*, which expands nodes only on a discrete 2D grid without considering vehicle orientation or turning radius. The output of the Trajectory Planning is a series of waypoints that represent the planned trajectory from the current vehicle pose to the next local waypoint.

6.1.2.2 Trajectory Optimization

Once the local trajectory has been planned, the next step is Trajectory Optimization. The Trajectory Optimization is responsible for optimizing the planned trajectory for smoothness and feasibility. The main idea of Trajectory Optimization is to blend the local trajectory at time t with time t-1 and time t-2. The main benefit of blending trajectory over time are to reduce sudden changes in trajectory that can lead to uncomfortable ride, reduce the risk of jitter pose estimation, reduce the risk of sensor noise, and improve the overall stability of the vehicle control.

The main idea of blending trajectory is doing α-β-γ filter for each trajectory by the time. The first step is to finding the neares point from current trajectory points to t-1 trajectory and t-2 trajectory. After that, we can do a weighted average for each point in current trajectory with the nearest point from t-1 trajectory and t-2 trajectory.

$$T_t^{\text{opt}} = \alpha\, T_t + \beta\, T_{t-1} + \gamma\, T_{t-2} \tag{6.1}$$

Equation 6.1 which inspired from [2] shows the standard weighted filter for α-β-γ. The T_t^{opt} is the optimized trajectory at time t, T_t is the planned trajectory at time t, T_{t-1} is the planned trajectory at time t-1, and T_{t-2} is the planned trajectory at time t-2. The α, β, and γ are the weight factors for each trajectory, where $\alpha + \beta + \gamma = 1$. In our implementation, we set $\alpha = 0.6$, $\beta = 0.3$, and $\gamma = 0.1$ to give more weight to the current trajectory while still considering the previous trajectories for smoothness. The output of Trajectory

Optimization is an optimized trajectory that will be used by Target Velocity and Steering Calculation to calculate the target velocity and steering angle for the vehicle.

6.1.2.3 Target Velocity and Steering Calculation

The final step of Local Navigation System is Target Velocity and Steering Calculation. The Target Velocity and Steering Calculation is responsible for calculating the target velocity and steering angle for the vehicle based on the optimized trajectory from Trajectory Optimization. The target velocity and steering angle are calculated using bicycle model kinematic. The first step is finding the angle to the next trajectory point based on lookahead distance. This can be done using equation 6.2.

$$\theta_{\text{direction}} = \tan^{-1}\left(\frac{y_{\text{Tp}} - y_{\text{position}}}{x_{\text{Tp}} - x_{\text{position}}}\right) - \theta_{\text{orientation}} \tag{6.2}$$

Where $(x_{\text{Tp}}, y_{\text{Tp}})$ is the next trajectory point, $(x_{\text{position}}, y_{\text{position}})$ is the current vehicle position, and $\theta_{\text{orientation}}$ is the current vehicle orientation. After that, we can calculate the target steering angle using bicycle model kinematic as shown in equation 6.3.

$$\delta_{\text{target}} = \tan^{-1}\left(\frac{2 \cdot L \cdot \sin(\theta_{\text{direction}})}{D_{\text{lookahead}}}\right) \tag{6.3}$$

Where δ_{target} is the target steering angle, L is the wheelbase of the vehicle, and $D_{\text{lookahead}}$ is the lookahead distance. The target velocity is calculated based on the curvature of the optimized trajectory and the maximum allowable lateral acceleration of the vehicle. The output of Target Velocity and Steering Calculation is the target velocity and steering angle that will be sent to the Safety System for execution.

6.2 Safety System

The main idea of Safety System is to ensure that the vehicle run normally. The Safety System consists of two main parts, which are ISO Implementation and Emergency Stop System.

6.2.1 ISO 26262 Implementation

There is standard ISO 26262 for Road autonomous vehicle functional safety. The ISO 26262 implementation is done to ensure that the vehicle operates safely and reliably in various driving conditions. There are list of condition and action that we have implemented based on ISO 26262 standard:

- If the LIDAR sensor fails, the vehicle will stop immediately and wait for manual intervention.
- If the camera sensor fails, the vehicle will stop immediately and wait for manual intervention.
- If the GPS sensor fails, the vehicle will switch to odometry-based navigation and reduce speed to a safe level.
- If the wheel velocity sensor fails, the vehicle will switch to GPS-based navigation and reduce speed to a safe level.
- If the vehicle deviates from the planned trajectory by more than a predefined threshold, the vehicle will slow down and attempt to replan the trajectory.
- If an obstacle is detected within a predefined distance from the vehicle, the vehicle will slow down or stop to avoid a collision.

6.2.2 Emergency Stop System

The main idea of Emergency Stop System is to double check the target velocity and steering angle from Local Navigation Estimation before sending the command to the vehicle's actuation system. We use 3d point cloud LIDAR to do that. The simple tangent circle scan applied to filter is there are any obstacle in the way that target velocity and steering angle will go through. If there are any obstacle detected in the way, the Emergency Stop System will reduce the target velocity to 0 and keep the steering angle to the last safe steering angle. The output of Emergency Stop System is the final target velocity and steering angle that will be sent to the vehicle's actuation system.

6.2.3 Ablation Study on Safety System

The ablation study of Safety Tests is done to evaluate the performance of the Safety System in various driving conditions by remove or add some feature condition. The Safety Tests are conducted in a controlled environment with predefined scenarios that simulate real-world driving situations. There are several scenarios that we have tested:

- Cloudy weather condition with low visibility.
- Multipath detection of GPS.
- Jitter pose estimation from Graph Based SLAM.

CHAPTER 7

REAL WORLD IMPLEMENTATION

7.1 Low Level System Summary

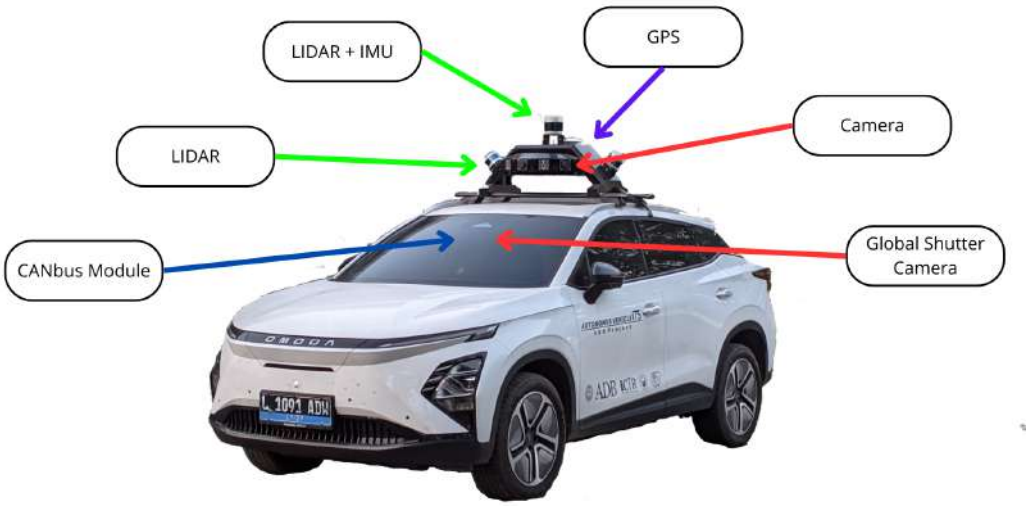

Figure 7.1. Sensor Implementation on Icar

Icar with new system was implemented by modifying the fully conventional vehicle. The modification includes adding a new set of sensors and managing the main CANbus. Figure 7.1 shows the sensor implementation on Icar. The sensors used in this system include 2x LIDAR, 1x LIDAR with IMU, GPS, 3x RGB Camera, and 1x global shutter camera.

Figure 7.2. Computing Hardware

Figure 7.2 shows the hardware to do all the program and computation. The green circle is the power supply unit to supply power to the main computer and all the sensor based on 7.1. The red circle is the main computer that installed inside the vehicle.

7.2 Implementation of Depth Image Creation

Based on figure 5.1, the first major step is Depth Image Creation. This step convert all LIDAR point cloud data to the single depth image based on camera intrinsic. Figure 7.3 shows the result of Depth Image Creation from LIDAR point cloud data.

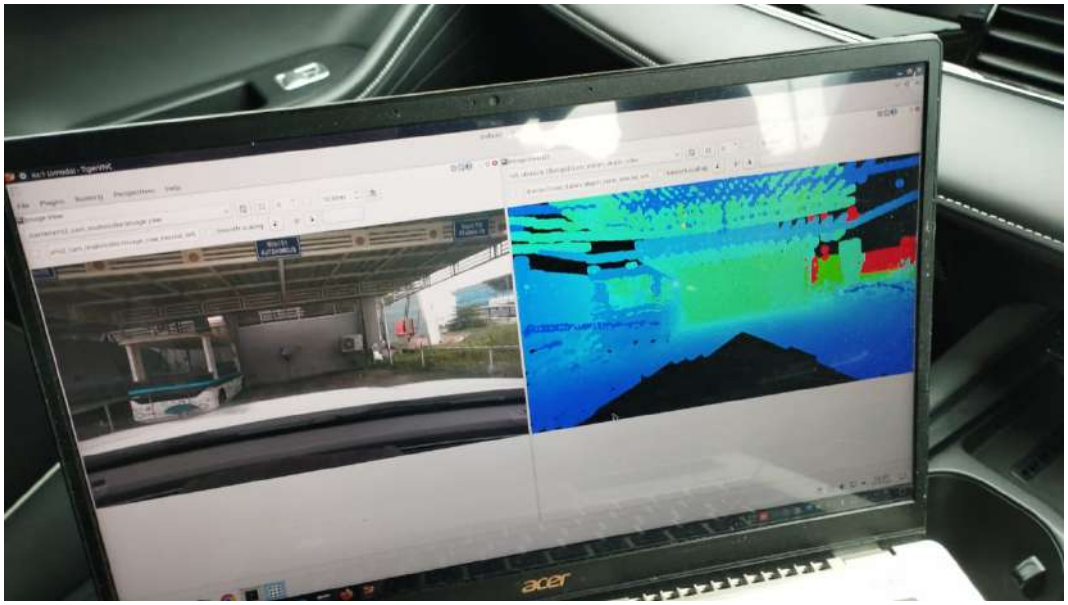

Figure 7.3. Result of Depth Image Creation

In the figure 7.3, the left image is the RGB image from the camera, and the right image is the depth image created from LIDAR point cloud data. The depth image painted on Jet colormap, where the closer the distance to the camera, the color will be more blue, and the farther the distance to the camera, the color will be more red.

7.3 Implementation of Road Segmentation

Once we have trained our model using our training strategy on 5.3.3, we can implement our model on to the vehicle. The first step is to load the model then inference the rgb image by that model. Figure 7.4 shows the result of Road Segmentation from our model.

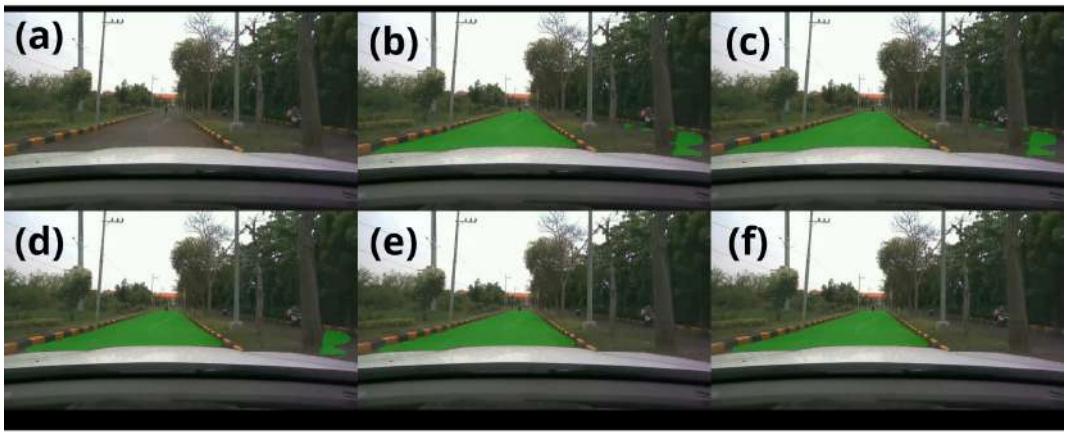

Figure 7.4. Result of Road Segmentation

In the figure 7.4, the sub image (a) is the RGB image from the camera, (b) is the raw Road Segmentation result from the model, (c) is the result of ROI cut process, (d) is the result of morphological operation process, (e) is the result of selecting the largest area, and (f) is the final Road Segmentation result after applying low pass temporal effect.

The last sub image (f) in figure 7.4 is the final Road Segmentation result that will be used on the next step, which is transforming the Road Segmentation to the base link frame by using depth image from 7.3. Figure 7.5 shows the result of Road Segmentation transformation to the base link frame.

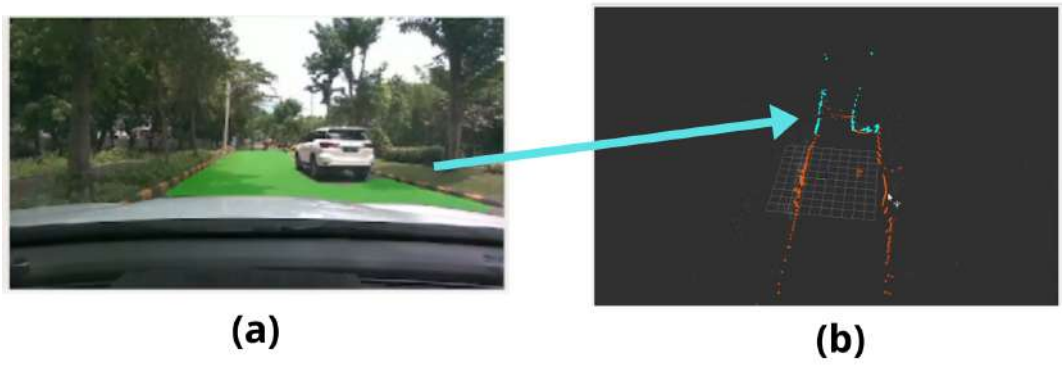

Figure 7.5. Result of Road Segmentation Transformation

In the figure 7.5, the sub image (a) is the segmented road image and (b) is the road point cloud laser scan in the base link frame. In sub image (b), the cyan point is the road point cloud that obtained from the transformation process.

7.4 Implementation of Graph Based SLAM

Based on figure 5.1 and figure 5.4, there are 5 inputs that going to Graph Based SLAM block: RGB Image, Depth Image from LIDAR, Road Laser Scan, GPS, and odometry. The GPS and odometry used to find and filter out nodes outside local-radius area for finding loop closures. Then, the RGB image and depth image from LIDAR used to find the loop closures by using Bag of Words method. The road laser scan used to calculate the constraint for GTSAM optimization process. The last step is doing pose fusion to produce high frequency pose estimation like we do before on [17].

The the first major step is synchronization. The synchronization has been done using approximate sync with delay tolerance 0.1 second. We have setup data queue size for synchronization up to 30, so it will be tolerance for various data update rate.

The second step is the image feature extraction. The strategy that we

used to do that is using ORB feature extraction. By combining it from the guess pose of GPS and odometry, we can create a Bag of Words that will be saved and proceed inside Nodes Processing. Figure 7.6 shows the result of ORB feature extraction.

Figure 7.6. Result of ORB Feature Extraction

The figure 7.6 shows the result of ORB feature extraction where the yellow points are the features that have been extracted using ORB algorithm. Those features will be converted into Bag of Words if the new node appear.

The next step is to do registration from the current node into saved nodes. The image registration using ORB and RANSAC. The ORB will be found the same feature from current image in current node into the previous saved Bag of Words in saved nodes. Figure 7.7 shows the result of image matching process.

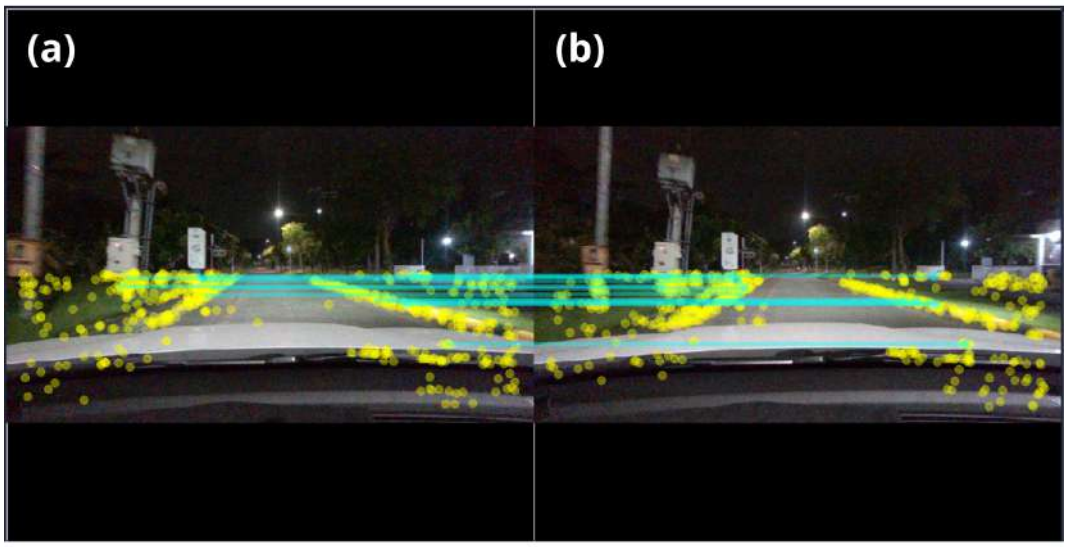

Figure 7.7. Result of ORB and RANSAC

Figure 7.7 shows the result of image matching process using ORB and RANSAC where the yellow points are the features for each images and the cyan line is the inliers detected and filtered using ORB and RANSAC. The sub image (b) is the current image in current node and the sub image (a) is the previous saved image in the database.

After we have found the inliers, the next step is thresholding the number of inliers found. If the number of inliers found more than the threshold, the loop closure hypothesis will be added. In the other hand, the strategy for the registration constraint in GTSAM optimization process is using ICP (Iterative Closest Point) algorithm between the current road laser scan and the previous saved road laser scan in the database.

The final output of GTSAM Optimization process is the optimized pose estimation. That optimized pose estimation will be fused with high frequency odometry to produce high frequency pose estimation using Kalman Filter. The strategy that we used is by setting the odometry as a prediction by doing a difference between the current odometry and the previous odometry. Then, the optimized pose estimation from GTSAM will be used as a correction measurement for Kalman Filter.

7.5 Implementation of Navigation System

The implementation of navigation system is divided into three parts. The first part is global navigation estimation. The second part is local navigation estimation. The last part is the safety system implementation.

The global navigation estimation is implemented by using the pose estimated from the implementation of graph based SLAM 7.4 and the 3rd party system that produces waypoints to target navigation. By doing a window filter for 20 m, we can filter the waypoint within 20 m radius from the estimated pose. After we have a filtered waypoints, the next step is to convert that waypoint into meter level coordinate based on base link using haversine formula. The result from that process will produce a trajectory called global trajectory. Figure 7.8 shows the result of global navigation estimation.

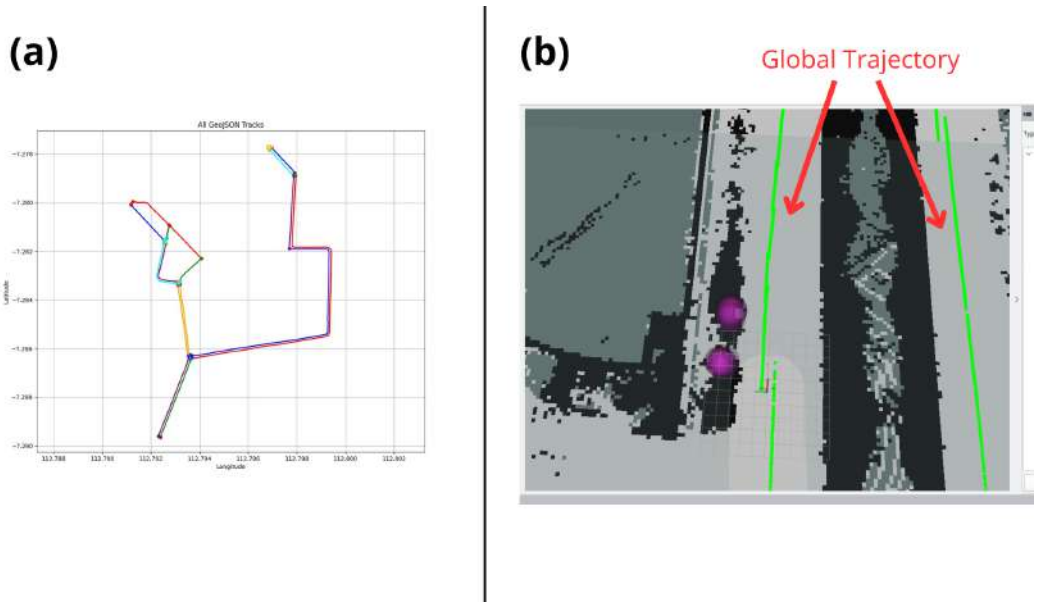

Figure 7.8. Result of Global Navigation Estimation

The sub image (a) in the figure 7.8 is the full path target that coming from 3rd party system. The sub image (b) is the global trajectory that will be used to compute next action. The global trajectory represented by the green line points inside sub image (b).

The next step is local navigation system that will produces a local

trajectory and target velocity and steering. Figure 7.9 shows the result of the local navigation estimation.

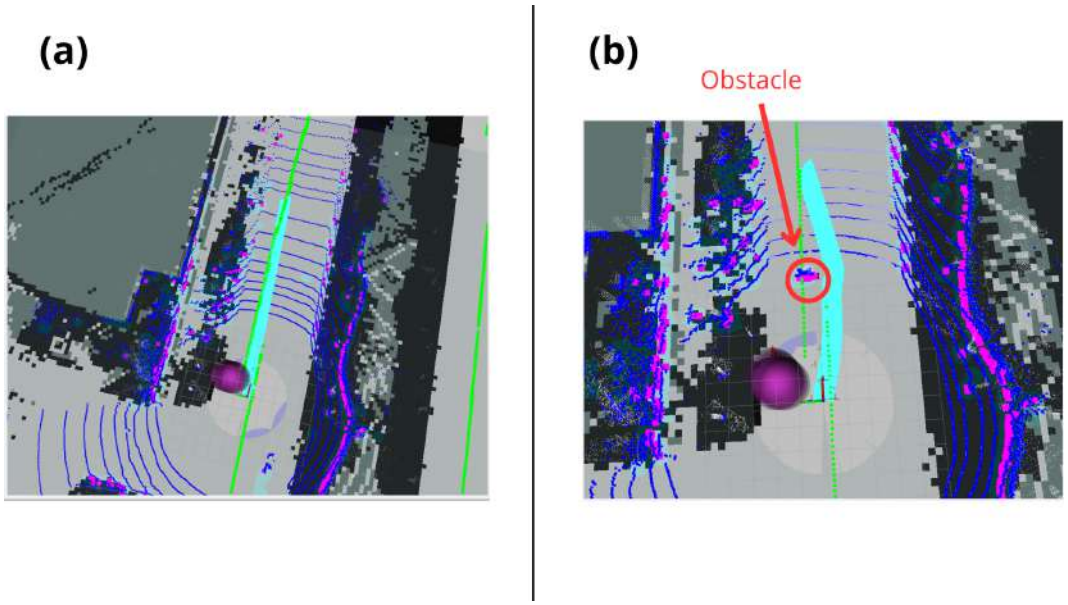

Figure 7.9. Result of Local Navigation Estimation

In the figure 7.9, both sub image (a) and (b) represent the result of local navigation estimation. The sub image (a) is the local navigation system that follow the generated global trajectory without any obstacles. The sub image (b) is the local navigation system that obstructed by an obstacle. From the figure 7.9, we can see that the local trajectory (the cyan line points) in sub image (b) is avoiding the obstacle (the red area) by planning a new local trajectory that avoid the obstacle.

CHAPTER 8
DISCUSSION ABOUT ROAD SEGMENTATION

8.1 Segmentation Accuracy

The segmentation accuracy is measured by using Intersection over Union (IoU) metric. The IoU metric is calculated by dividing the intersection area between the predicted segmentation and the ground truth segmentation with the union area between the predicted segmentation and the ground truth segmentation. Table 8.1 shows the IoU segmentation accuracy result at different epoch during the training process.

Table 8.1. IoU Segmentation Accuracy Result

Epoch	**IoU**
150	0.7697
200	0.7760
300	0.7831
400	**0.7928**
500	0.7867

Table 8.1 shows that the best IoU segmentation accuracy is achieved at epoch 400 with IoU value of 0.7928. This model will be used for the implementation on the vehicle.

It is considered that the IoU calculation for accuracy measurement is adequate enough because in real world application, there is a dimention called time. The IoU calculation for table 8.1 is calculated by selecting several random images that not included in the training dataset. But in real world application, the model will receive a continuous stream of images from the camera. So, we cannot trust only on the IoU calculation for several random images. We need

to consider the temporal effect in real world application. The next section will discuss about the Importance of temporal effect in Road Segmentation application.

8.2 The Importance of Temporal Effect

Like we discuss before, in real world application, the model will receive a continuous stream of images from the camera. So, we need to consider the temporal effect in real world application. The temporal effect that we imply in our model is using GRU (Gated Recurrent Unit) that will always save the global h state from the previous frame to the next frame. Figure 8.1 shows the comparison between with GRU temporal effect and without temporal effect in our Road Segmentation model.

Figure 8.1. With GRU Temporal Effect vs Without Temporal Effect

In the figure 8.1, the sub image (a) is the result of raw Road Segmentation using GRU and sub image (b) is the result of raw Road Segmentation without using GRU. All that mask captured without being post processed so we can analyze the effect of GRU temporal effect only. From the figure 8.1, we can see that there is a hole called False Positive in sub image (b) because it does not use GRU temporal effect. That False Positive hole can be dangerous in

real world application because it can mislead the navigation system to think that the area is not drivable area.

Sometimes, even though we have used GRU temporal effect, there is a chance that the False Positive hole still appears in the Road Segmentation. Figure 8.2 shows another example of False Positive hole that appear even though we have used GRU temporal effect.

Figure 8.2. Another With GRU Temporal Effect vs Without Temporal Effect

Same as figure 8.1, in the figure 8.2, the sub image (a) is the result of raw Road Segmentation using GRU and sub image (b) is the result of raw Road Segmentation without using GRU. All that mask captured without being post processed so we can analyze the effect of GRU temporal effect only. From the figure 8.2, we can see that there is a hole called False Positive in sub image (a) even though it smaller than the False Positive hole in sub image (b). To handle this case, we need to apply the post processing step to filter out the False Positive hole in the Road Segmentation result. The next section will discuss about the important of post processing in our Road Segmentation application.

8.3 The Importance of Post Processing

Like we discuss before, we have a problem called False Positive hole that appear in the Road Segmentation result even though we have used GRU temporal effect. We think it was a flickering problem that sometime happen in real world application. To handle this problem, we found out to apply a low pass filter over time. The idea is calculate per pixel value in frame t by the effect of previous frame t-1. The equation is shown below:

$$S_t(x, y) = \alpha \cdot S_{t-1}(x, y) + (1 - \alpha) \cdot S_t(x, y) \tag{8.1}$$

where S is the segmentation mask, t is the current frame, t-1 is the previous frame, (x,y) is the pixel coordinate, and α is the low pass filter coefficient that we set to 0,1. After calculate pixel value, we divide it to binary value by thresholding it with 0.5 value. Figure 8.3 shows the effect of post processing using low pass temporal filter.

Figure 8.3. Raw vs Low Pass Temporal Filtered Road Segmentation

In the figure 8.3, the sub image (a) is the raw Road Segmentation result from the model, and (b) is the Road Segmentation result after applying low pass temporal filter. From the figure 8.3, we can see that the False Positive

hole that appear in sub image (a) has gone in sub image (b) after applying low pass temporal filter. This post processing step is important to make the Road Segmentation result more stable and reliable for the navigation system.

Not only used to temporally filter the Road Segmentation result, we also use several post processing step spatially to make the Road Segmentation result more accurate. The spatial post processing step that we use are ROI cut, morphological operation, and selecting the largest area. Figure 7.4 shows the effect of spatial post processing step in our Road Segmentation application. The sub image (d) to (e) in figure 7.4 shows the effect of morphological operation and selecting the largest area post processing step. From the figure 7.4, we can see that the spatial post processing step is important to remove the small unwanted segmented area that can mislead the navigation system.

CHAPTER 9

RESULT OF GRAPH BASED SLAM

9.1 Result of GPS Only vs Graph Based SLAM

The main idea of this section is to compare the result of using the GPS only for navigation system and using the proposed graph based SLAM for navigation system. The detail of this section will be explained in the sub section 9.1.1 and 9.1.2.

9.1.1 Pose Estimation Accuracy

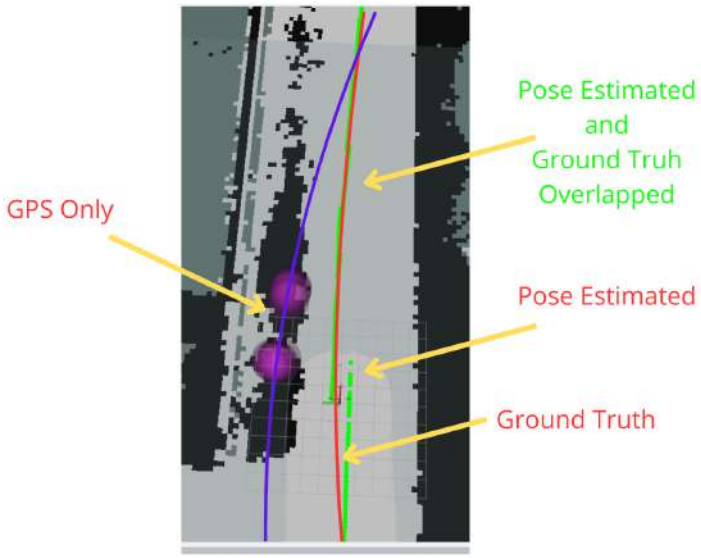

Figure 9.1. Accuracy Test of GPS Only vs Graph Based SLAM

Figure 9.1 shows the accuracy test result of GPS only vs Graph Based SLAM. The purple line shows the GPS only pose estimation, while the green line shows the Graph Based SLAM pose estimation. The red line is the ground truth path that we used to compare the accuracy of both GPS only and Graph Based SLAM. From the figure 9.1, we can see that the GPS only has not converged to the ground truth path, while the Graph Based SLAM has

converged to the ground truth path.

From the figure 9.1, we can see that the GPS only pose estimation is far better from the ground truth path compared to the Graph Based SLAM pose estimation. In other hand, There is a single jitter that happen on the Graph Based SLAM pose estimation usually because of the sensor synchronization problem for each sensor and the conversion from wheel odometry to the base link car frame.

9.1.2 Pose Estimation Precision

Table 9.1. Precision Test of GPS Only vs Graph Based SLAM

GPS x	**GPS y**	**SLAM x**	**SLAM y**
0.2	0.2	0.00	0.00
0.4	0.2	0.10	0.00
0.2	0.1	0.00	0.00
0.4	0.7	0.01	0.00
1.9	1.2	0.00	0.00
1.2	0.8	0.02	0.00
1.2	0.6	0.00	0.05

Table 9.1 shows the precision test result of GPS only vs Graph Based SLAM. The precision test is done by stationary the vehicle in one position and record the pose estimation from GPS only and Graph Based SLAM for 7 times. From the table 9.1, we can see that the GPS only has a precision error up to 2.24 meters, while the Graph Based SLAM has a precision error up to 0.10 meters. The standard deviation also can be calculated and have a result 0.69 in GPS only estimation and 0.03 for the graph based slam. This result shows that the Graph Based SLAM has a much better precision than GPS only for navigation system. That can happen because we used the wheel odometry as the main prior for the pose estimation in Graph Based SLAM, so if the vehicle is stationary, the odometry will not change and the pose estimation will be

more precise.

The little the standard deviation value, the more stable the system is. So, from the standard deviation result that we have calculated before, we can conclude that the Graph Based SLAM system is more stable than GPS only system for navigation system.

CHAPTER 10

ABLATION STUDIES

10.1 Ablation Study on Navigation System

Based on the list of tests in 6.2.3, we have done several ablation studies on the navigation system to find the robustness of the proposed navigation system.

When the weather become cloudy and low visibility, the road detection still work well because we have added the dataset with cloudy and low visibility condition during the training process. Figure 10.1 shows the result of road detection in cloudy and low visibility condition.

Figure 10.1. Result of Road Detection in Cloudy and Low Visibility Condition

Figure 10.1 shows that the road detection still work well in cloudy and low visibility condition. The green area is the detected road area.

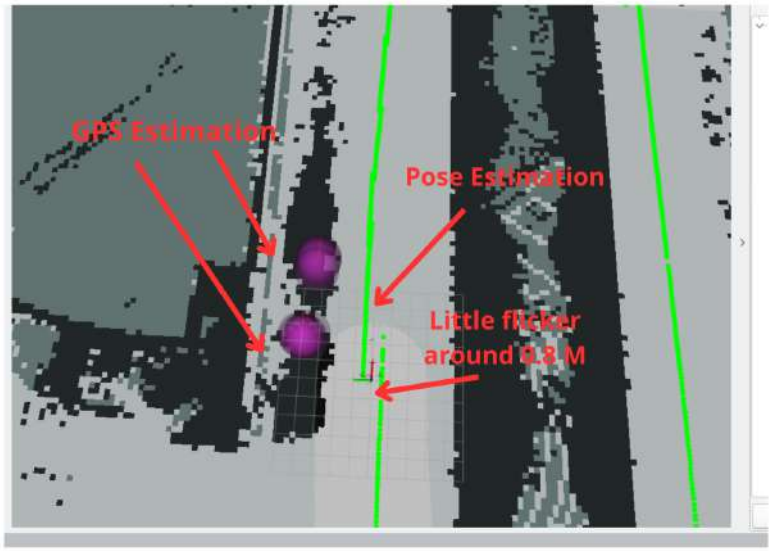

Figure 10.2. Result of Pose Estimation in Multipath GPS Condition

Figure 10.2 shows the result of pose estimation in multipath GPS condition. The multipath GPS condition represented by the sphere purple area. The pose estimation represented by the green line. From the figure 10.2, we can see that the pose estimation still work well in multipath GPS condition because even through there still a little jitter on the pose estimation around 0.5 meter.

When there is a jitter pose estimation happen like in figure 10.2, the navigation system still navigate well because have applied an α-β-γ filter to smooth the trajectory.

This page is intentionally left blank

CHAPTER 11

CONCLUSION

We have done implemented a new navigation system for autonomous vehicle using Graph Based SLAM and Machine Learning. The Graph Based SLAM used to improve the pose estimation accuracy. The Machine Learning used to do Road Segmentation simultaneously.

We have successfully implemented the Graph Based SLAM to improve the pose estimation from the maximum error 2.24 meters to 0.10 meters. Another improvement is in the precision of the pose estimation that improved from 0.69 meters to 0.03 meters.

We have also successfully implemented the Road Segmentation using Machine Learning that can run in real-time with the best IoU 0.7928. Not only that, we do temporal filtering such as embedding GRU in the network and doing post processing low pass filter frame by frame to improve the segmentation result stability.

This page is intentionally left blank

Bibliography

[1] Nabila Abraham and Naimul Mefraz Khan. A novel focal tversky loss function with improved attention u-net for lesion segmentation. In *2*019 IEEE 16th International Symposium on Biomedical Imaging (ISBI 2019), pages 683–687, 2019.

[2] Alex Becker. *K*alman Filter from the Ground Up. kalmanfilter.net, 3rd edition, 2024. Accessed from kalmanfilter.net.

[3] Martin BROSSARD and Silvère BONNABEL. Learning wheel odometry and imu errors for localization. In *2*019 International Conference on Robotics and Automation (ICRA), pages 291–297, 2019.

[4] Cesar Cadena, Luca Carlone, Henry Carrillo, Yasir Latif, Davide Scaramuzza, José Neira, Ian Reid, and John J. Leonard. Past, present, and future of simultaneous localization and mapping: Toward the robust-perception age. *I*EEE Transactions on Robotics, 32(6):1309–1332, 2016.

[5] Frank Dellaert. Factor graphs and gtsam: A hands-on introduction. *G*eorgia Institute of Technology, 2012. Available at `https://gtsam.org`.

[6] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *D*eep Learning. MIT Press, Cambridge, MA, 2016.

[7] Mohinder S. Grewal, Lawrence R. Weill, and Angus P. Andrews. *G*lobal Navigation Satellite Systems, Inertial Navigation, and Integration. John Wiley & Sons, Hoboken, NJ, 2nd edition, 2013.

[8] Rudolf E. Kalman. A new approach to linear filtering and prediction problems. *J*ournal of Basic Engineering, 82(1):35–45, 1960.

[9] Elliott D. Kaplan and Christopher J. Hegarty. *U*nderstanding GPS/GNSS: Principles and Applications. Artech House, Norwood, MA, 3rd edition, 2017.

[10] Shaharyar Ahmed Khan Tareen and Rana Hammad Raza. Potential of sift, surf, kaze, akaze, orb, brisk, agast, and 7 more algorithms for matching extremely variant image pairs. In *2*023 4th International Conference on Computing, Mathematics and Engineering Technologies (iCoMET), pages 1–6, 2023.

[11] Samer Khanafseh, Birendra Kujur, Mathieu Joerger, Todd Walter, Sam Pullen, Juan Blanch, Kevin Doherty, Laura Norman, Lance de Groot, and Boris Pervan. Gnss multipath error modeling for automotive applications. In *P*roceedings of the 31st International Technical Meeting of the Satellite

Division of The Institute of Navigation (ION GNSS+ 2018), pages 1573–1589, Miami, Florida, 2018. Institute of Navigation (ION).

[12] Kourosh Khoshelham and Sander Oude Elberink. Accuracy and resolution of kinect depth data for indoor mapping applications. *S*ensors, 12(2):1437–1454, 2012.

[13] Mathieu Labbé and François Michaud. Rtab-map: A real-time appearance-based mapping approach for large-scale and long-term slam. *I*EEE Transactions on Robotics, 36(4):1199–1210, 2019.

[14] Mathieu Labbé and François Michaud. Rtab-map as an open-source lidar and visual simultaneous localization and mapping library for large-scale and long-term online operation. In *J*ournal of Field Robotics, volume 36, pages 416–446. Wiley Online Library, 2019.

[15] Stefan Leutenegger, Margarita Chli, and Roland Y. Siegwart. Brisk: Binary Robust Invariant Scalable Keypoints. In *2*011 IEEE International Conference on Computer Vision (ICCV), pages 2548–2555. IEEE, 2011.

[16] S. Macenski. Smac planner: Hybrid-a*, 2d-a*, and lattice-based global planners for ros 2 navigation. `https://github.com/ros-navigation/navigation2/tree/main/nav2_smac_planner`, 2021. Accessed: 2025-XX-XX.

[17] A. W. Maulana, R. Dikairono, A. Zaini, M. I. Zazuli, Muhtadin, and M. H. Purnomo. Hybrid navigation system for autonomous vehicle using extended rtab-map and reduced FAST-SCNN. In *2*025 International Conference on Advanced Mechatronics, Intelligent Manufacture And Industrial Automation (ICAMIMIA), Surabaya, Indonesia, 2025.

[18] Pratap Misra and Per Enge. *G*lobal Positioning System: Signals, Measurements and Performance. Ganga-Jamuna Press, Lincoln, MA, 2006.

[19] Tom M. Mitchell. *M*achine Learning. McGraw-Hill, New York, 1997.

[20] Brian Paden, Michal Čáp Câmara, Sze Zheng Yong, Dmitry Yershov, and Emilio Frazzoli. A survey of motion planning and control techniques for self-driving urban vehicles. *I*EEE Transactions on Intelligent Vehicles, 1(1):33–55, 2016.

[21] Arunava Chakraborty Poudel, Stephan Liwicki, and Roberto Cipolla. Fast-scnn: Fast semantic segmentation network. 2019.

[22] Rajesh Rajamani. *V*ehicle Dynamics and Control. Springer, Boston, MA, 2nd edition, 2011.

[23] Pandu Surya Tantra, Rudy Dikairono, Hendra Kusuma, Muhtadin, and Tasripan. Automated lidar-based dataset labelling method for road image segmentation in autonomous vehicles. In *2*024 International Conference on Green Energy, Computing and Sustainable Technology (GECOST), pages 1–5, 2024.

[24] Sebastian Thrun, Wolfram Burgard, and Dieter Fox. *P*robabilistic Robotics. MIT Press, Cambridge, MA, 2005.

[25] A. Vinay, Avani S. Rao, Vinay S. Shekhar, Akshay C. Kumar, and K. N. Balasubramanya Murthy. Feature extraction using orb-ransac for face recognition. *P*rocedia Computer Science, 70:174–184, 2015. Presented at 4th International Conference on Eco-friendly Computing and Communication Systems (ICECCS 2015).

[26] Greg Welch and Gary Bishop. An introduction to the kalman filter. `https://www.cs.unc.edu/~welch/kalman/`, 2006. Online tutorial, University of North Carolina at Chapel Hill.

[27] Ami Woo, Baris Fidan, W.W. Melek, and R. Buehrer. Localization for autonomous driving. pages 1051–1087, 01 2019.

[28] Moh. Ismarintan Zazuli, Rudy Dikairono, Djoko Purwanto, Muhtadin, and Muhammad Lukman Hakim. Sensor fusion system for localization of autonomous car. In *2*024 International Conference on Green Energy, Computing and Sustainable Technology (GECOST), pages 1–5, 2024.

[29] Juyong Zhang, Yuxin Yao, and Bailin Deng. Fast and robust iterative closest point. *I*EEE Transactions on Pattern Analysis and Machine Intelligence, 44(7):3450–3466, 2022.

This page is intentionally left blank

AUTHOR BIOGRAPHY

Personal Information

Name	:	Azzam Widan Maulana
Place of Birth	:	Jember
Date of Birth	:	June 4, 2002
Address	:	Arif Rahman Hakim Street No. 14A, Keputih, Sukolilo, Surabaya

Educational Background

2024–Present	:	Master's Program (M.Sc.), Department of Electrical Engineering, Faculty of Intelligent Electrical and Informatics Technology, Institut Teknologi Sepuluh Nopember (ITS)
2020–2024	:	Bachelor Program (B.Eng.), Department of Computer Engineering, Faculty of Intelligent Electrical and Informatics Technology, Institut Teknologi Sepuluh Nopember (ITS)
2017–2020	:	SMAN 1 Jember
2014–2017	:	SMPN 3 Jember
2008–2014	:	SDN Kaliwates 2 Jember
2006–2008	:	TK Miftahul Ulum Kaliwates Jember

List of Publications

1. Muhtadin, A. W. Maulana, R. Dikairono and A. Zaini, "Omnivision Calibration on Mobile Robot Using Machine Learning," *2024 International Conference on Computer Engineering, Network, and Intelligent Multimedia (CENIM)*, Surabaya, Indonesia, 2024, pp. 1–8, doi: 10.1109/CENIM64038.2024.10882712.

Research History

1. Multicast Communication System on IRIS Robot 2022
2. Natural Ball Handling System on IRIS Robot 2022
3. Omnivision Camera Calibration on Mobile Robot Using Machine Learning 2023
4. Robot Positioning System Using Gaussian Model on IRIS Robot 2023
5. Empty Goal Detection System Using Omnivision Camera on IRIS Robot 2023
6. Content Management System and User Interface for RAISA ITS Robot 2024
7. Custom Operating System for Autonomous Vehicle and IRIS Robot 2024
8. Empty Goal Detection System Using Stereo Depth Camera on IRIS Robot 2024
9. Ball Detection System Using Spatial Image Classification on IRIS Robot 2024
10. Hardware Communication System Using Layer 2 Communication for Autonomous Vehicle and IRIS Robot 2024
11. Hardware Communication System Using CAN-bus for Autonomous Vehicle and IRIS Robot 2024
12. End to End Machine Learning System for Autonomous Vehicle 2025
13. Fleet Management System for AMR (Autonomous Mobile Robot) 2025

Other Background

Before venturing into the world of robotics and programming, the author was involved in music. As a musician, he mastered various musical instruments such as guitar, bass, and keyboard. In addition to playing instruments, he was also active in the field of mixing and mastering. His hobby of signal processing sparked an interest in learning how computers process signals. This passion led him to pursue a degree in Computer Engineering at ITS. Moreover, he delved into robotics to understand how computers and hardware operate, enabling him to perform advanced digital signal processing techniques.